

Argmaxing and complexity

Easy solutions to hard problems

Martín Soto

September 8, 2023

LIs over actions are solved

Say Ω 's Logical Inductor $\mathbb{O}$ only observes $\mathbf{A}$'s actions, as a syntactical variable. That is, Ω doesn't know $\mathbf{A}$'s code from the start, and instead $\mathbb{O}$ will slowly figure it out (implicitly applying a simplicity prior).

Consider the algorithm $\mathbf{A}$ that, when dropped at the root of a finite tree, calculates completely which finite sequence of actions maximizes reward in this tree, and then enacts it. This involves simulating $\mathbb{O}$'s response to several different action sequences, as well as computing the outcomes of all logical coins it will face. That is, being completely updateful. This $\mathbf{A}$ is PlanCDT (treating $\mathbb{O}$ as nothing more than causally manipulable levers in the environment), and it maximizes reward (on each particular tree) by definition.¹

When the tree is infinite, since we assume there are no branches with unboundedly high value, all far enough rewards will be arbitrarily low, and so we can perform the same procedure to within ε reward accuracy.

That's in a very obvious sense the best we can do (it's easy to see that perfect accuracy in infinite arbitrary trees is uncomputable).

LIs over math are not solved?

Let's go back now to our original framing: Ω knows $\mathbf{A}$'s code, and checks $\mathbb{O}$'s opinions on the logical sentences " $\mathbf{A}(\mathcal{Q}) = a$ ". What prevents us from implementing the above solution?

We'd like to choose as $\mathbf{A}$ the algorithm that computes how much each $\mathbf{A}'$ scores in the current tree, and then just implement it (that is, run it and copy its actions). Of course, this is not computable, because of non-halting of some $\mathbf{A}'$ (which didn't happen above because of each action policy being just given as an explicit sequence). We can stipulate how long we want to run this search for, and then pick the best available option, but that won't give us reward guarantees as before (it's always possible that we've missed an $\mathbf{A}'$ that scores way better, because of quirks of $\mathbb{O}$).

¹Don't be fooled by the "Agent simulates Predictor" flavour of this setup. Here, since the only influence is through actions, $\mathbf{A}$ will just find the best such policy. If that involves "acting as if it hadn't simulated the Predictor" (whatever that means), it will just do that.

Even granted that, we are sneaking in an imperfect analogy: It's not true that "**A** searching for **A'** and then running it" is the same algorithm as "**A'**". And indeed, since $\mathbb{O}$ looks at the algorithm, there's no guarantee they will be treated equally in finite rounds.² That is, we have sneakily hard-coded a computable way to take counterfactuals ("what would happen if I were **A'**"), but it doesn't completely correspond to reality. In reality, **A**'s counterfactuals should be "what would happen if, by the end of the search I'm currently carrying out, my best option were **A'**". This is a logical counterfactual, and so the only way we have to deal with it is letting **A** run an LI $\mathbb{A}$ for a long time, and then stop at some arbitrarily pre-determined point to use that knowledge.

There are worries here about the Happy Dance problem, but I don't yet know how they resolve. Imagine $\mathbb{A}$ soon comes to the well-founded conclusion "I will only choose the completely cooperative algorithm **C** if the environment turns out to be very cooperative". Thus, $\mathbb{A}(U \mid \forall \mathcal{Q}(\mathbf{A}(\mathcal{Q}) = \mathbf{C}(\mathcal{Q})))$ ends up being very high (because that formula is evidence for the world being very cooperative), even in the face of lots of evidence about the world being un-cooperative. But wait... then $\mathbb{A}$ should notice this dynamic, and so it will be less true that "I will only choose the completely cooperative algorithm **C** if the environment turns out to be very cooperative". These become complicated questions about $\mathbb{A}$ predicting future states.³

Optimal performance

In the former case, we realize an obvious notion of optimal performance (per tree). In the latter, it's clearly impossible for any **A** to be optimal in all trees.

Why is this the case? Because in the former case, all influence on $\mathbb{O}$ is screened off by the policy chosen, and so **A** can freely argmax over those. While in the latter, this "naturalistic boundary" has been pulled further back, and now the algorithm **A** itself has indelible consequences. Now, it is us, from the outside, who are free to think about possible **A** algorithms, and argmax over them. But any **A** will already be constrained by embeddedness if it tries to do the same.

This embeddedness can be unfair to **A** in specific situations, so we'd instead like some "on average" (across trees) notion of optimality. The following one would seem easy to obtain.

On-average optimality: Say $\bar{U}_n(\mathbf{A})$ is the average reward **A** obtains across all finite decision trees with at most n nodes, logical coins of at most length n , and reward at most n at each node. We'd like an **A** such that $\bar{U}_n(\mathbf{A})$ eventually dominates all other $\bar{U}_n(\mathbf{A}')$ (as a function of n). Alternatively, we could restrict our trees further to ensure the limit of this value is finite (for example, by demanding rewards grow slowly enough, and dividing by n or n^2), and then ask for $\lim_n \bar{U}_n(\mathbf{A})$ to be maximal.

In fact, I think this optimality is already attained by the above "**A** searching for **A'** (search power increasing with tree size)" (the naive way of taking counterfactuals), by an argument as in 2.

²But it is true that, granted some weak assumptions (like Ω not forever using early $\mathbb{O}$ states), they will be treated equally in the limit. That's because $\mathbb{O}$ will learn that they output the same actions, and it's only ever queried about output actions.

³I also had the intuition that we needn't worry about " $\mathbb{A}$ thinking for too long" (as with Logical Coins), because it is always thinking about one of its future states (before arriving at that state that **A** uses to take the decision), and so will not reach certainty about it. But it is actually possible in some particular situations (even if not achievable in general) for $\mathbb{A}$ to have strong certainty about some of its values at much later times.

This notion of optimality can also be translated to infinite trees. In those we again don't have reward guarantees, but I think we can compensate for those by having computational resources grow, and taking that limit (thus "eventually encompassing all computable programs").

I'd also like to try and find the "good version" of "using an LI $\mathbb{A}$ to implement the more realistic logical counterfactual" (although it might involve solving too many LI problems), and think of optimality notions more overfit to that algorithm.

The case for introducing computational complexity

Thus, both cases above seem to be pretty satisfactorily argmaxed away. This "encompassing all computable programs" makes me suspicious that maybe there's not so much to say about merely computable solutions, and we should instead look into the finer details of computational resources to find "the kinds of cognition that are actually natural or favored".

In a dream world, I'd like to get something like "under computational constraints, we can understand optimal performance as approximating LIs in this way". This seems far because of our limited understanding of LI behavior... but maybe that shouldn't be the main bottleneck.

Appendix: Picking flowers

I'm interested in how all the above interacts with "truly one-shot" updatelessness (since we've lost a bit of that by using $\mathbb{O}$).

In the second case, using a regular LI, we can formalize the flower problem as the following decision tree: $\mathbb{O}$ runs for a long while, looks at the code for $\mathbf{A}$, and decides whether it would pick flowers were it to be created (which is a logical counterfactual), and then $\mathbf{A}$ is created accordingly. Here it is true that $\mathbf{A}$'s code more legibly cooperating plays to its advantage. I hoped that if we take the limit of $\mathbb{O}$ thinking for infinitely long, then any program that "will clearly cooperate" gets built. But there's a problem: $\mathbb{O}_n$ has to decide whether $\mathbf{A}$ cooperates in the world where it is created, that is, the world where $\mathbb{O}_n$ had a high probability of this self-referential query. That is, we'll consult $\mathbb{O}_n$'s opinion on the sentence

$$S \equiv \mathbb{O}_n(S) \geq p \rightarrow \mathbf{A}() = C$$

and check whether it's $\geq p$. By limit LI self-trust, it seems like by choosing large enough n (and high p) we'll have high

$$\mathbb{O}_n(\mathbb{O}_n(S) \geq p \rightarrow S)$$

and thus of course also high $\mathbb{O}_n(S)$, regardless of $\mathbf{A}$. I don't yet know if we can formalize the flower problem while avoiding this.

On the contrary, in the first case, with an action-based LI, there clearly is no way to formalize the problem. There is no action that corresponds to "your algorithm showing you will cooperate". Indeed, because of the above mentioned shift in position of the boundary between embeddedness and non-embeddedness, there is no way for "something to happen based on $\mathbf{A}$'s existence, or before $\mathbf{A}$ can take a move".